

Trabajo Práctico 2: CSS — Flexbox y Grid

Versión didáctica — fundamentación, resultados de aprendizaje, enunciados en prosa, errores frecuentes, rúbrica y autoevaluación.

Tiempo estimado: 30 minutos por actividad.

1. Sentido formativo del trabajo práctico

Este trabajo práctico te lleva de estilar elementos sueltos a organizar la página. Hasta ahora dabas color y fuente a cada elemento; a partir de acá vas a decidir dónde se ubica cada cosa: en fila, en columna, en una grilla. Vas a usar los dos sistemas de layout modernos de CSS: Flexbox, pensado para acomodar elementos en una dimensión (una fila o una columna), y Grid, pensado para acomodarlos en dos dimensiones (filas y columnas a la vez).

La idea central que vas a apropiarte es la del contenedor que ordena a sus hijos: al declarar `display: flex` o `display: grid` en un elemento, ese elemento pasa a controlar cómo se distribuyen los elementos que tiene adentro. Vos no movés cada hijo uno por uno: definís reglas en el contenedor —dirección, alineación, separación, columnas— y el navegador ubica todo por vos.

¿POR QUÉ ESTO IMPORTA?

Flexbox y Grid son la base del diseño de interfaces en la web actual: tarjetas, barras de navegación, galerías, catálogos. Aprender desde el primer día a pensar en contenedores que distribuyen su contenido —en vez de empujar elementos a mano— es un hábito profesional que hace tus diseños ordenados, flexibles y fáciles de adaptar, y que te va a acompañar en toda la carrera.

Qué tenés que resolver y entregar

Este trabajo práctico consiste en armar una página de catálogo con un producto destacado (usando Flexbox) y una grilla de productos (usando Grid). Esto es lo que entregás:

PRODUCTO A ENTREGAR

1. Los archivos `index.html` y `style.css`, con el CSS resolviendo el layout de la página de catálogo.
2. La página debe abrir en el navegador mostrando el producto destacado ordenado con Flexbox y el catálogo en una grilla de 3 columnas con Grid.
3. Los estilos deben usar `display: flex` en la tarjeta destacada y `display: grid` en el catálogo, y respetar la paleta de colores indicada.

Cada punto marcado con ENTREGABLE es parte obligatoria de lo que se corrige.

Todo lo demás del documento (fundamentación, errores frecuentes, rúbrica) es material de apoyo; no se entrega, pero se evalúa a través de la calidad de tu código.

2. Marco conceptual

Concepto	Qué significa en tu página
Contenedor y elementos	El contenedor lleva <code>display: flex</code> o <code>display: grid</code> ; sus hijos directos son los elementos que se distribuyen.
Flexbox	Sistema de una dimensión: acomoda los hijos en una fila o una columna.
Dirección del flex	<code>flex-direction</code> define el eje: <code>row</code> (fila, por defecto) o <code>column</code> (columna).

Concepto	Qué significa en tu página
Alineación	<code>align-items</code> alinea los elementos en el eje transversal (por ejemplo, centrarlos verticalmente en una fila).
Grid	Sistema de dos dimensiones: acomoda los hijos en filas y columnas a la vez.
Columnas de la grilla	<code>grid-template-columns</code> define cuántas columnas y de qué ancho (por ejemplo, <code>repeat(3, 1fr)</code>).
Separación	<code>gap</code> define el espacio entre los elementos, tanto en flex como en grid.

3. Resultados de aprendizaje esperados

Ordenados según Bloom: desde reproducir una distribución con flex o grid hasta decidir qué sistema conviene para cada parte del diseño.

Resultado de aprendizaje	Nivel Bloom
Reconocer la diferencia entre un contenedor flex y uno grid, y el rol de sus propiedades.	Comprender
Acomodar elementos en fila y en columna con Flexbox, alineándolos y separándolos.	Aplicar
Distribuir elementos en una grilla de columnas iguales con Grid.	Aplicar
Separar los elementos de forma consistente con <code>gap</code> .	Aplicar
Decidir cuándo usar Flexbox y cuándo Grid según la estructura a construir.	Analizar

4. Requerimientos transversales (obligatorios)

Estas reglas aplican a todo el trabajo.

El contenedor manda. Las propiedades de layout (`flex-direction`, `align-items`, `grid-template-columns`, `gap`) se declaran en el contenedor que lleva `display: flex` o `display: grid`, no en los hijos. Fundamento: es el contenedor el que distribuye a sus elementos.

Flex para una dimensión, Grid para dos. La tarjeta destacada se resuelve con Flexbox; el catálogo, con Grid. Fundamento: usar el sistema adecuado a cada caso simplifica el código y el resultado.

Separación con `gap`. El espacio entre elementos se logra con `gap`, no con márgenes sueltos en cada hijo. Fundamento: `gap` da una separación uniforme y fácil de ajustar en un solo lugar.

Paleta respetada. Los colores usados son exactamente los de la paleta indicada. Fundamento: seguir una paleta definida da consistencia visual, tal como se trabaja en un proyecto real.

5. Actividades

A continuación se describen las dos actividades que debés resolver, redactadas para que las leas de corrido y sepas exactamente qué construir. Ambas se aplican sobre la misma página de catálogo.

Actividad 1 — Flexbox

Foco: acomodar elementos en fila y en columna con un contenedor flex; es el primer contacto con el layout flexible.

Vas a ordenar la tarjeta del producto destacado. En `.featured-card` usás `display: flex` para ubicar la imagen y la info en una fila, las alineás verticalmente con `align-items: center`, y separás la imagen de la info con `gap`.

Después, ordenás la info en columna. Dentro de `.featured-info` usás `display: flex` con `flex-direction: column` para acomodar el título, la descripción, el precio y el botón uno debajo del otro.

Tarea de prueba (lo que debés programar y entregar):

ENTREGABLE En `.featured-card`: `display: flex`, `align-items: center` y `gap` para poner imagen e info en fila, alineadas y separadas.

ENTREGABLE En `.featured-info`: `display: flex` con `flex-direction: column` para apilar título, descripción, precio y botón.

Actividad 2 — Grid

Foco: distribuir varios elementos en una grilla de columnas iguales con un contenedor grid.

Vas a armar el catálogo como una grilla. En `.grid` usás `display: grid` con `grid-template-columns: repeat(3, 1fr)` para mostrar los productos en 3 columnas iguales, y separás los productos entre sí con `gap`.

Para que la grilla tenga contenido, el catálogo debe incluir al menos 6 tarjetas de producto (`.product-card`), cada una con su imagen, su nombre y su precio, de modo que la grilla se llene en dos filas de tres.

Tarea de prueba (lo que debés programar y entregar):

ENTREGABLE En `.grid`: `display: grid` con `grid-template-columns: repeat(3, 1fr)` y `gap` para separar los productos.

ENTREGABLE Incluir al menos 6 `.product-card` (con imagen, nombre y precio) para llenar la grilla.

Paleta de colores a usar

Usá exactamente estos valores. La columna “Muestra” te deja ver el color aplicado.

Elemento	Propiedad	Color	Muestra
header, footer	background-color	#2c3e50 azul oscuro	
header, footer	color	#ffffff blanco	
.featured-card / .product-card	background-color	#ffffff blanco	
.price / botón	color / background-color	#2c9d5f verde	
body	background-color	#f0f2f5 gris muy claro	

! 6. Errores frecuentes y cómo detectarlos

Cada error revela un malentendido conceptual concreto en esta etapa inicial.

ERROR 1 — Poner las propiedades de layout en el hijo y no en el contenedor

Síntoma: nada se acomoda; los elementos siguen apilados como sin flex ni grid.

Causa conceptual: se escribió `align-items` o `grid-template-columns` en un hijo, pero esas propiedades solo funcionan en el contenedor que tiene `display: flex/grid`.

Cómo detectarlo: verificá que `display: flex` o `display: grid` esté en el contenedor (`.featured-card`, `.grid`), no en las tarjetas de adentro.

ERROR 2 — Confundir flex-direction con la disposición esperada

Síntoma: el título, la descripción y el precio quedan en fila cuando deberían estar en columna (o al revés).

Causa conceptual: se olvidó `flex-direction: column` (el valor por defecto es `row`), o se lo puso donde iba fila.

Cómo detectarlo: si los elementos de `.featured-info` no quedan uno debajo del otro, revisá que tenga `flex-direction: column`.

ERROR 3 — La grilla no muestra 3 columnas

Síntoma: los productos aparecen en una sola columna o en un número de columnas distinto al esperado.

Causa conceptual: falta `display: grid`, o `grid-template-columns` no dice `repeat(3, 1fr)`, o se aplicó al elemento equivocado.

Cómo detectarlo: confirmá que `.grid` tenga `display: grid` y `grid-template-columns: repeat(3, 1fr)`, y que las tarjetas sean sus hijos directos.



7. Rúbrica de evaluación

Cada criterio se evalúa en tres niveles. Usala también como guía: apuntá al nivel Avanzado antes de entregar.

Criterio	Inicial	Suficiente	Avanzado
Flexbox en la tarjeta	No usa flex o no ordena nada.	Usa <code>display: flex</code> pero sin alinear o separar.	Imagen e info en fila, <code>align-items: center</code> y <code>gap</code> aplicados.
Columna con flex	La info no se apila.	Apila la info con algún detalle suelto.	<code>.featured-info</code> en columna con <code>flex-direction: column</code> .
Grid en el catálogo	No usa grid o queda en una columna.	Usa grid pero sin 3 columnas iguales.	<code>repeat(3, 1fr)</code> con <code>gap</code> , productos en 3 columnas.
Contenido de la grilla	Menos de las tarjetas pedidas.	Incluye tarjetas pero incompletas.	6+ <code>.product-card</code> con imagen, nombre y precio.
Colores y coherencia	No aplica la paleta.	Aplica algunos colores de la paleta.	Paleta respetada; página coherente y ordenada.



8. Autoevaluación (antes de entregar)

¿Puse `display: flex` en `.featured-card` y no en sus hijos?

¿La imagen y la info quedan en fila, centradas verticalmente con `align-items: center` y separadas con `gap`?

¿`.featured-info` apila título, descripción, precio y botón con `flex-direction: column`?

¿`.grid` muestra los productos en 3 columnas iguales con `repeat(3, 1fr)` y `gap`?

¿Podría explicar por qué usé Flexbox para la tarjeta y Grid para el catálogo?



9. Síntesis: herramientas de CSS para este práctico

Necesidad	Herramienta de CSS
Activar el layout flexible	<code>display: flex</code> en el contenedor
Poner en fila o en columna	<code>flex-direction: row / column</code>
Alinear en el eje transversal	<code>align-items: center</code>

Necesidad	Herramienta de CSS
Activar el layout en grilla	<code>display: grid</code> en el contenedor
Definir columnas iguales	<code>grid-template-columns: repeat(3, 1fr)</code>
Separar los elementos	<code>gap</code>

Checklist de entrega

- ☐ `.featured-card` usa `display: flex` con `align-items: center` y `gap`.
- ☐ `.featured-info` usa `display: flex` con `flex-direction: column`.
- ☐ `.grid` usa `display: grid` con `grid-template-columns: repeat(3, 1fr)` y `gap`.
- ☐ El catálogo tiene al menos 6 `.product-card` con imagen, nombre y precio.
- ☐ Los colores usados coinciden exactamente con la paleta indicada.
- ☐ El producto destacado y el catálogo se ven ordenados en el navegador.

Código inicial

Punto de partida para no arrancar de cero. Completá donde dice TODO.

```
index.html
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Mi Tienda - Catálogo</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

  <header>
    <h1>Mi Tienda</h1>
  </header>

  <main>
    <section class="featured">
      <h2>Producto destacado</h2>
      <div class="featured-card">
        
        <div class="featured-info">
          <h3><!-- TODO: nombre del producto --></h3>
          <p><!-- TODO: descripción --></p>
          <span class="price"><!-- TODO: precio --></span>
          <button>Comprar</button>
        </div>
      </div>
    </section>

    <section class="catalog">
      <h2>Catálogo</h2>
      <div class="grid">
        <!-- TODO: al menos 6 .product-card con img, h3 y span.price -->
      </div>
    </section>
  </main>

  <footer>
    <p>&copy; 2026 Mi Tienda</p>
  </footer>
```

```
</body>
</html>
```

style.css

```
body {
  font-family: Arial, sans-serif;
  margin: 0;
  background-color: #f0f2f5;
}

/* Actividad 1: Flexbox */
.featured-card {
  /* TODO: display flex, align-items, gap */
}

.featured-info {
  /* TODO: display flex, flex-direction column */
}

/* Actividad 2: Grid */
.grid {
  /* TODO: display grid, grid-template-columns, gap */
}
```